CSE5026: Foundation and Frontiers of Cognitive Science

Language and Syntax (II)

Yang Xu 徐炀

Department of Computer Science and Engineering

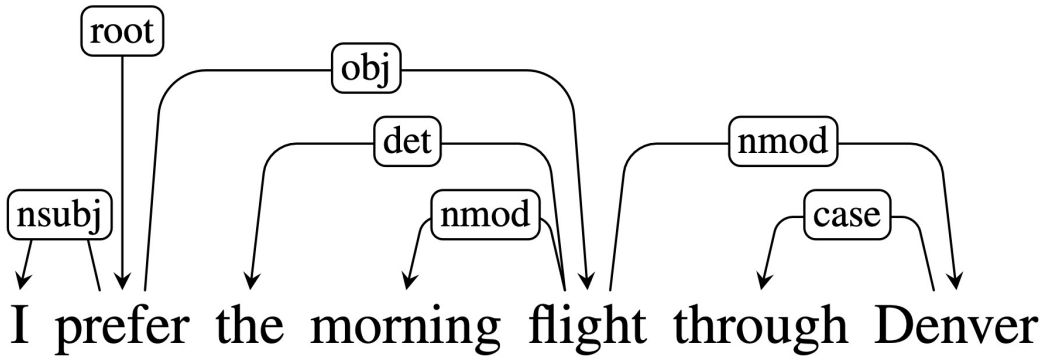
Southern University of Science and Technology

Overview

- Dependency Grammars 依存语法
- Transition-Based Dependency Parsing 依存句法解析
- Relation to Cognitive Science

Dependency Grammars

- Different from context-free grammars (CFG) and constituency grammars
- **Dependency Grammars** describe syntactic structure of a sentence solely in terms of directed grammatical *relations between words*



arc: *n.* 弧线

Labeled arcs from **heads** to **dependents**

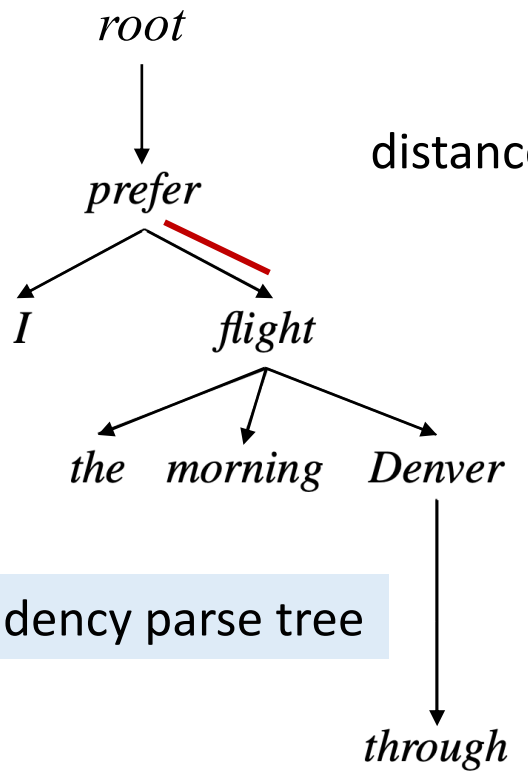
$prefer \xrightarrow{\text{nsubj}} I$ *prefer* is the head of *I*

$prefer \xrightarrow{\text{obj}} flight$ *flight* is the dependent of *prefer*

$root \rightarrow prefer$ *root* is the head of *prefer* and also the head of the entire structure (root of the tree)

Compare to Context-Free Grammars

- Head-dependent relations **directly** encode important information that is often **buried** in the more complex constituency parses (by CFG)

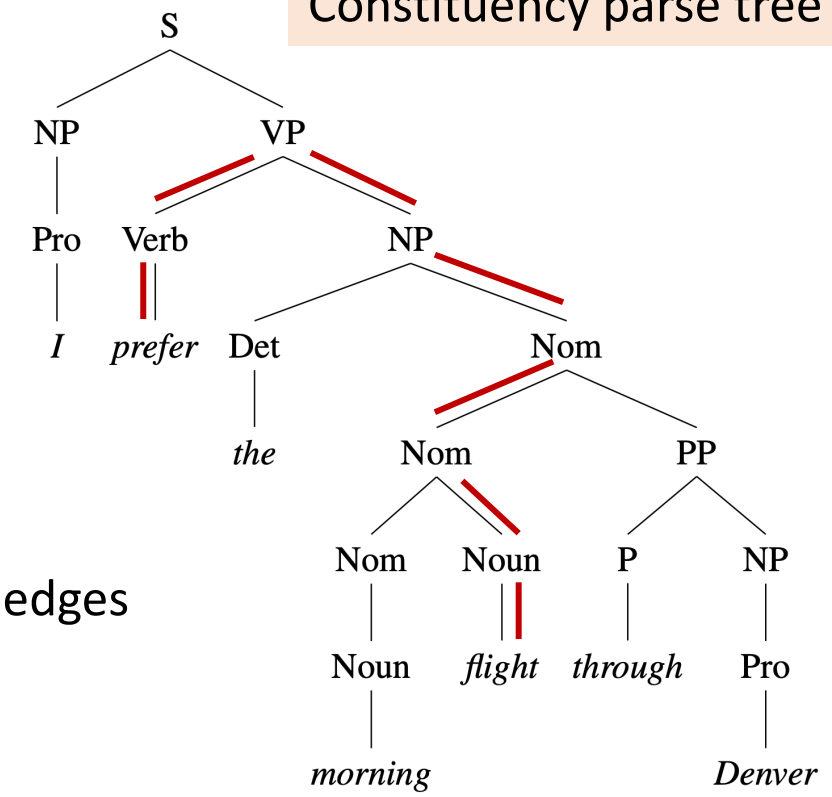


dependency parse tree

prefer flight

distance(Verb, Obj) = 1 edge

Constituency parse tree



prefer flight

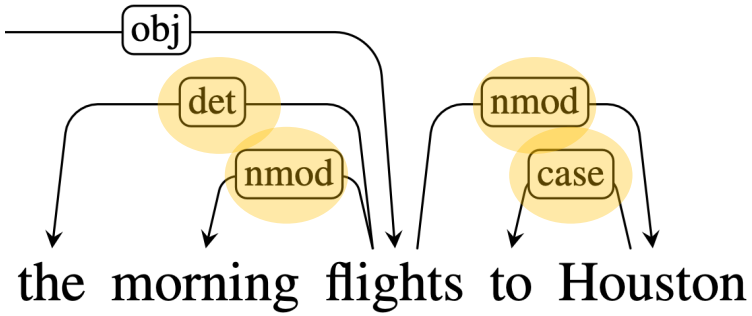
distance(Verb, Obj) = 7 edges

Dependency Relations

- Dependency relations consist of a head and dependent
- The ***head*** plays the role of the *central organizing word*, and the ***dependent*** as a modifier
- The relations can be classified based on the grammatical function:
- such as *subject*, *direct object*, *indirect object*

Dependency Relations (Examples)

- **Modifier** relations (修饰):

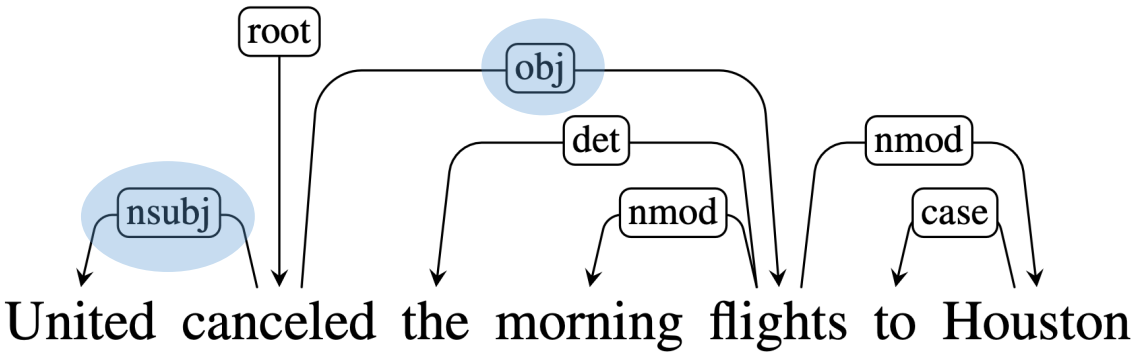


describing the ways that words can modify their heads

NMOD, DET, and CASE denote modifiers of the nouns *flights* and *Houston*

Dependency Relations (Examples)

- **Clausal** relations (从句):

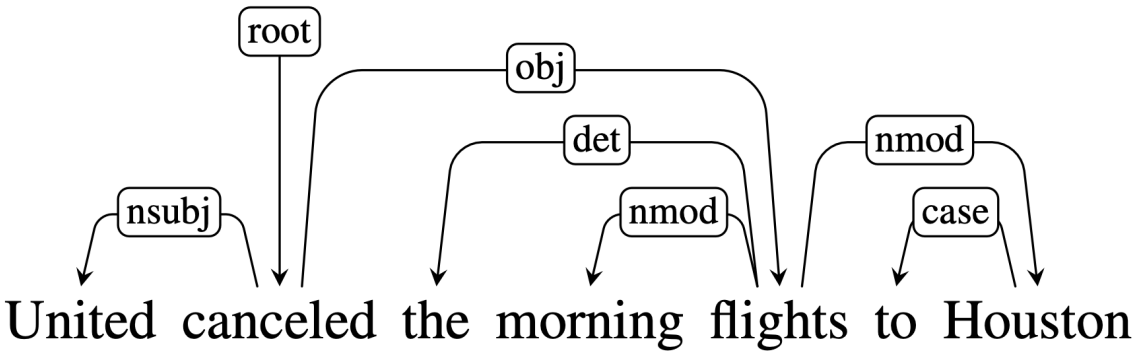


clausal relations describe syntactic roles with respect to a predicate (谓词 often a verb)

NSUBJ and OBJ identify the subject and direct object of the predicate *cancel*

Graph Formalization of Dependency

- A dependency structure can be represented as a directed graph $G = (V, A)$, with a set of vertices V, and a set of ordered pairs of vertices A, called **arcs**.



V: root, canceled, United, flights, …

$arcs$: root → canceled,

canceled $\xrightarrow{nsubj}$ United,

canceled $\xrightarrow{obj}$ flights, …

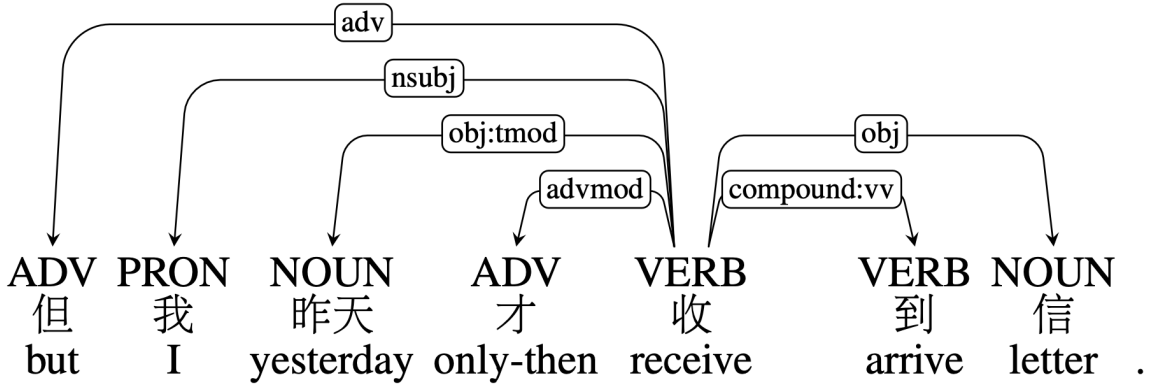
Graph Formalization of Dependency

- A dependency tree satisfies the following constraints:
- 1. There is a single root node that has no incoming arcs
- 2. Each vertex has exactly one incoming arc, except for the root
- 3. There is a unique path from the root node to each vertex in V

- With these constraints, each word has single head; a word can have multiple dependents.

Dependency Treebanks

- Treebanks play important role in training and evaluating dependency parsers, and for linguistic studies
- Treebanks are created by: human annotators; hand-corrected from the output of a parser; *translated* from constituency treebanks
- Largest open community project: The **Universal Dependencies (UD)** project
  - 200+ dependency treebanks in 100+ languages



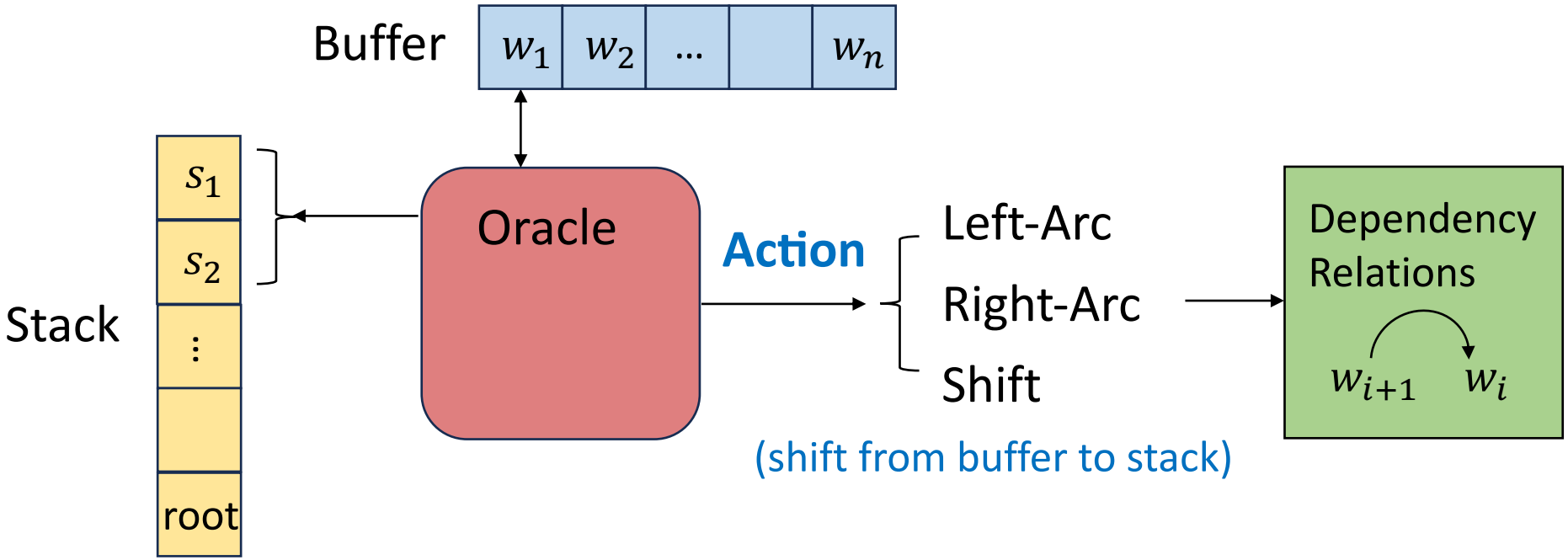
但我昨天才收到信 “But I didn’t receive the letter until yesterday”

Overview

- Dependency Grammars
- **Transition-Based Dependency Parsing**
  - Generic Parsing Process - Arc Standard Approach
  - Generating Training Data
  - Implementation
- Relation to Cognitive Measures

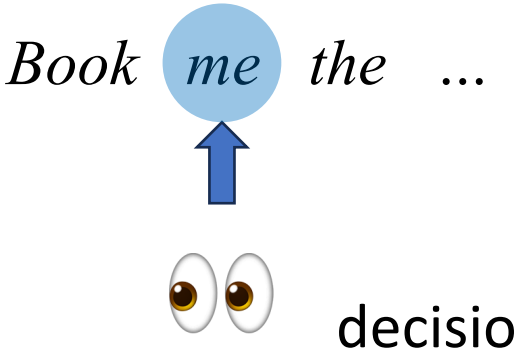
Transition-Based Dependency Parsing

- An architecture that draws on *shift-reduce* parsing (for designing programming language parsers)
- Key components: **stack**, **buffer**, and **oracle**.



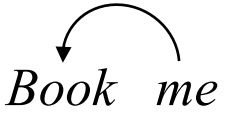
Transition actions

- The transition **actions** corresponds to the intuitive actions when we read a sentence in a single pass and try to create a dependency tree



decision:

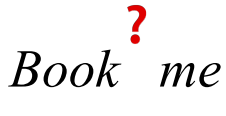
- ! Assign current word as the ***head*** of some previously seen word



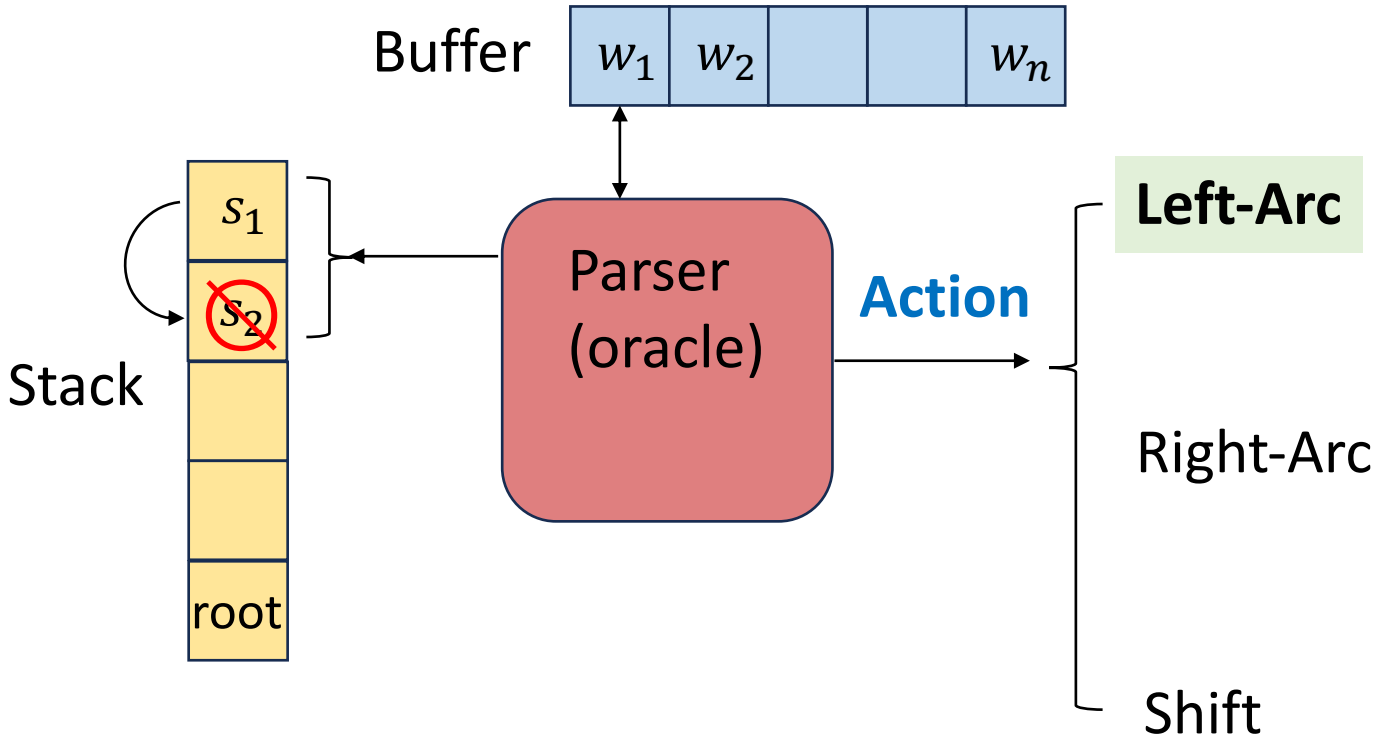
- ! Assign some previously seen word as the ***head*** of current word



- ? Not sure, so postpone the decision and store it for later processing



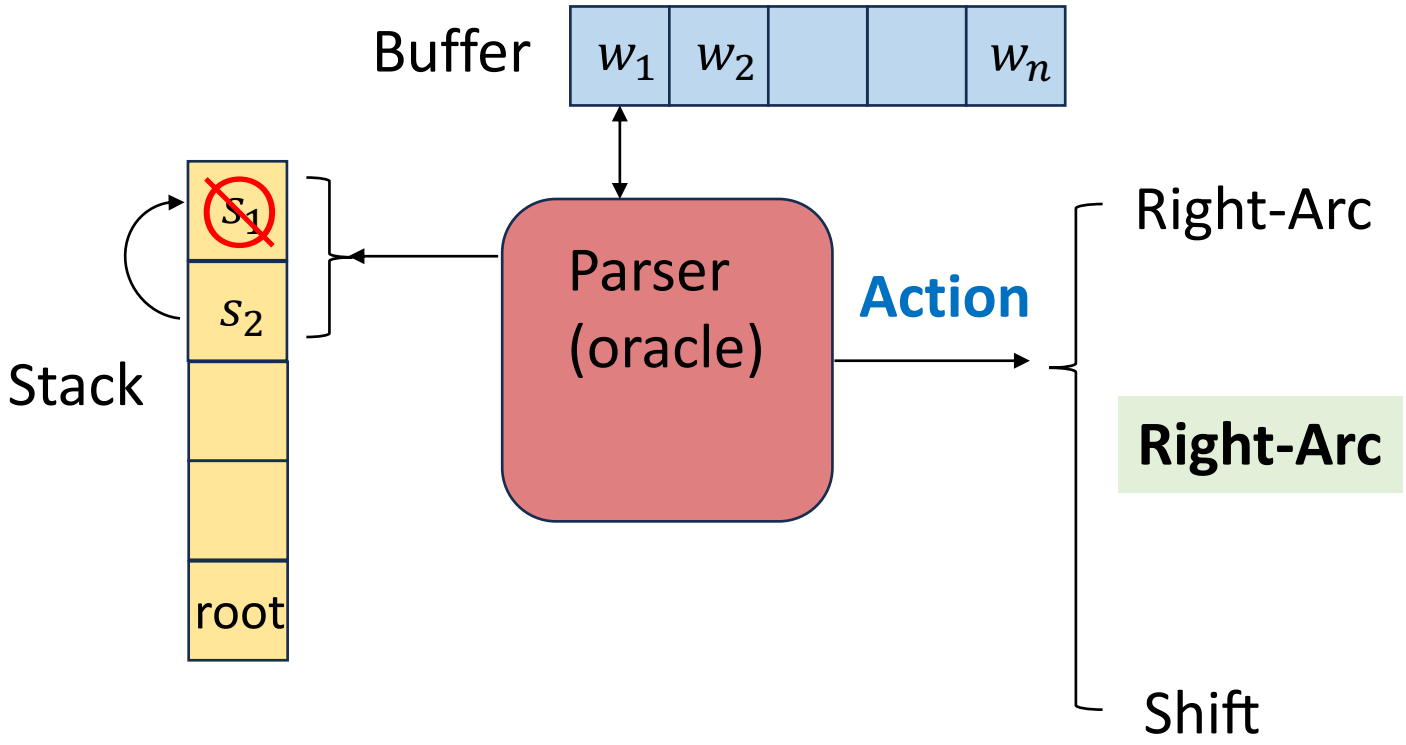
Formalize the transition actions



Assert $s_1 \rightarrow s_2$
(s_1 is head and s_2 is dependent)
Remove s_2

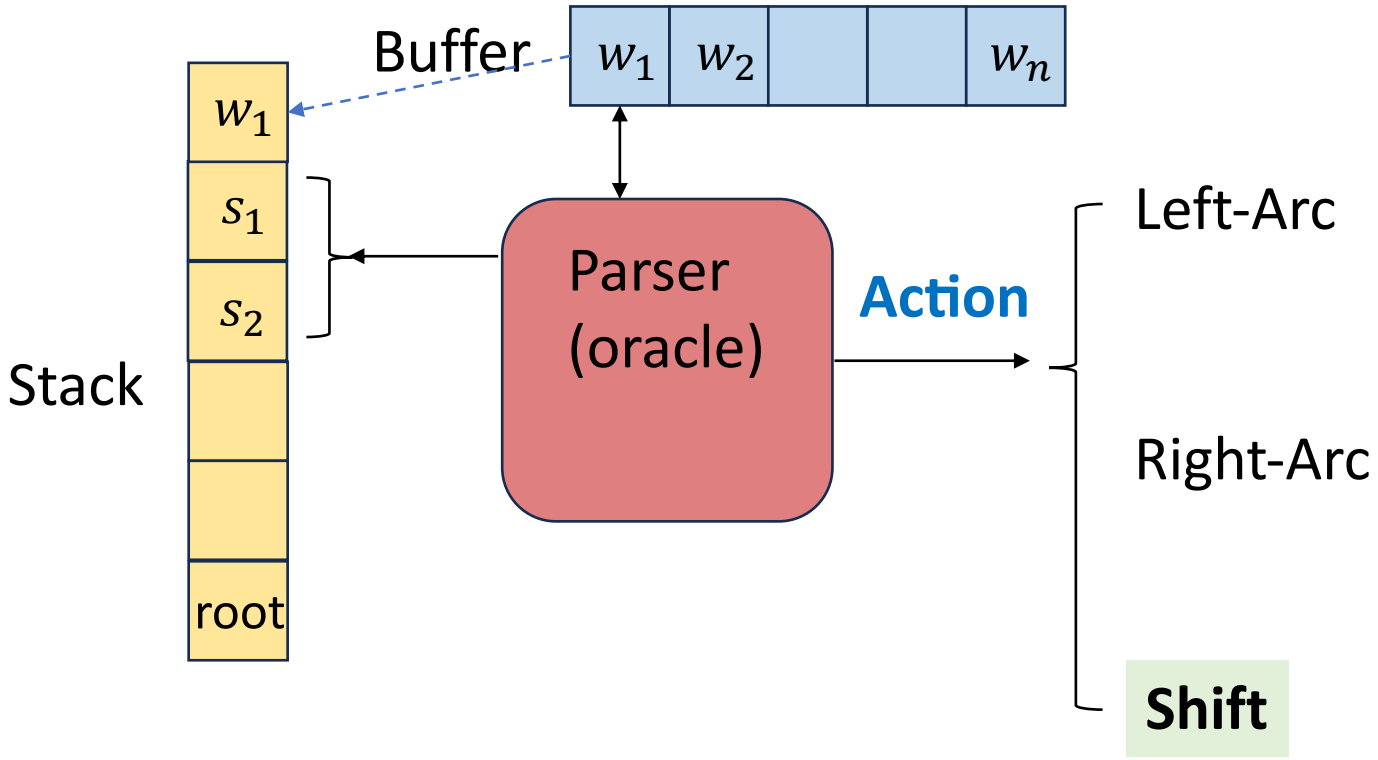
Because each word has exactly one income arc

Formalize the transition actions



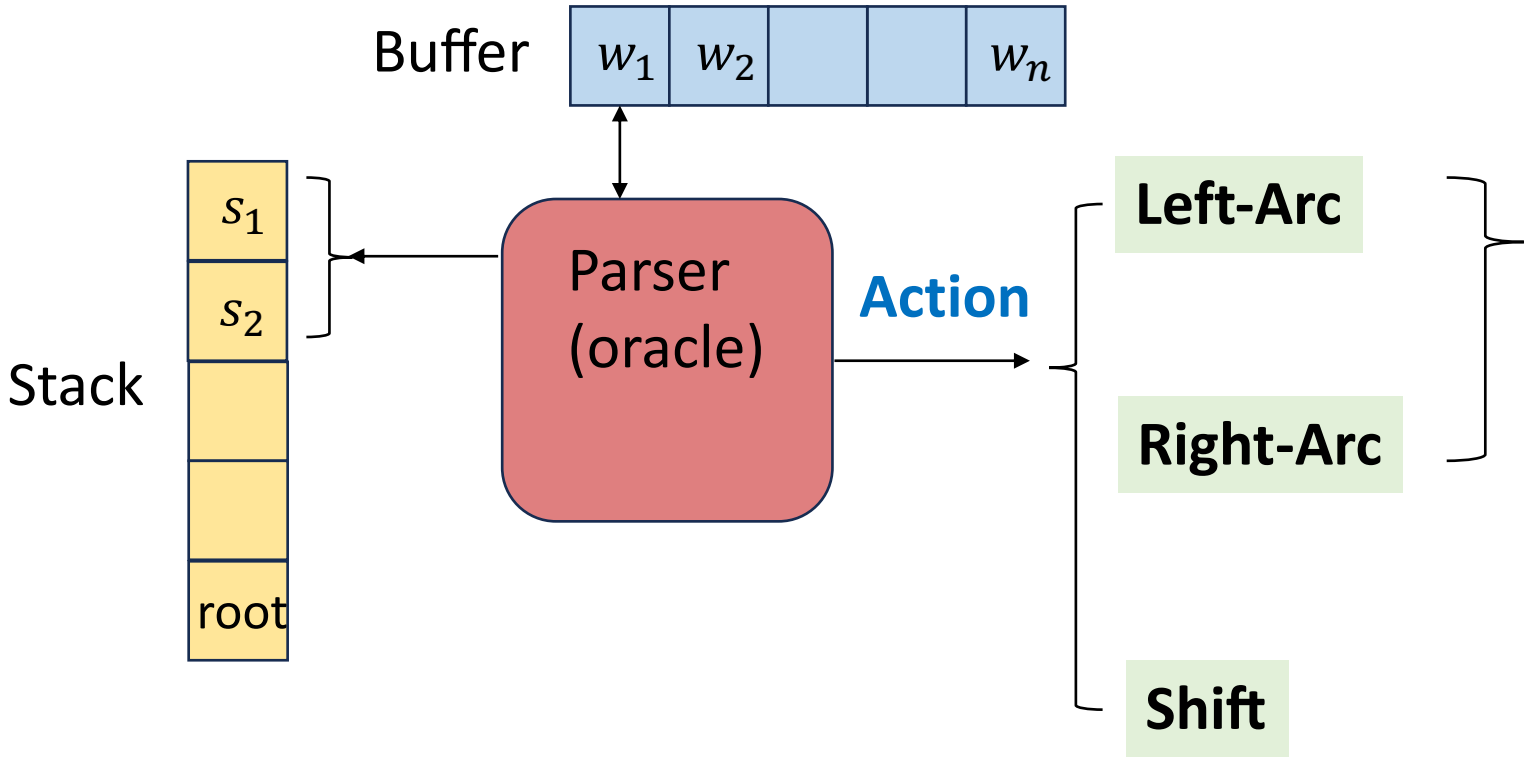
Assert $s_2 \rightarrow s_1$
(s_2 is head and s_1 is dependent)
Remove s_1

Formalize the transition actions



Remove the word from the front of **buffer** and push it to **stack**

Formalize the transition actions



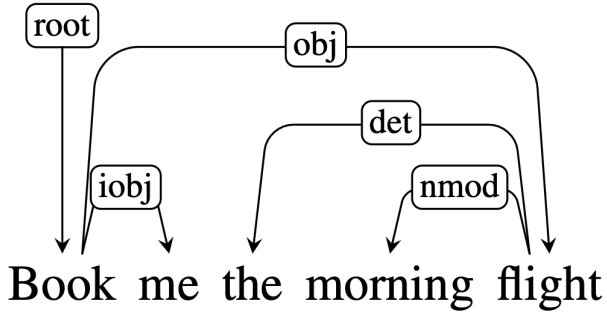
Reduce operations:
"reduce" means combining elements on stack

Parsing Example

Input sentence: *Book me the morning flight*

Initial state:
- stack: [**root**]
- buffer: [book, me, the, morning, flight]
- relations: []

Ground truth:



Add relation (book → me)

Step	Stack	Buffer	Action	Relation Added
0	[root]	[book, me, the, morning, flight]	Shift	
1	[root, book]	[me, the, morning, flight]	Shift	
2	[root, book, me]	[the, morning, flight]	Right-Arc	(book → me)

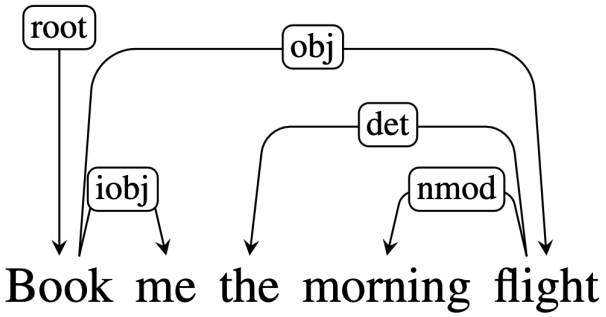
Pop *me* from the stack

Assign *book* as the head of *me*

Parsing Example

Input sentence: Book me the morning flight

Goal:



Step	Stack	Buffer	Action	Relation Added
0	[root]	[book, me, the, morning, flight]	Shift	
1	[root, book]	[me, the, morning, flight]	Shift	
2	[root, book, me]	[the, morning, flight]	Right-Arc	(book → me)
3	[root, book]	[the, morning, flight]	Shift	
4	[root, book, the]	[morning, flight]	Shift	
5	[root, book, the, morning]	[flight]	Shift	
6	[root, book, the, morning, flight]	[]	Left-Arc	(morning ← flight)

Assign *flight* as the head of *morning*

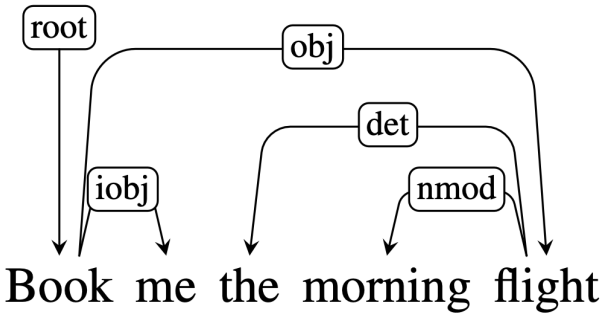
Add relation

Pop *morning* from the stack

Parsing Example

Input sentence: Book me the morning flight

Goal:



Step	Stack	Buffer	Action	Relation Added
6	[root, book, the, morning, flight]	[]	Left-Arc	(morning ⟵ flight)
7	[root, book, the, flight]	[]	Left-Arc	(the ⟵ flight)

Pop *the* from the stack

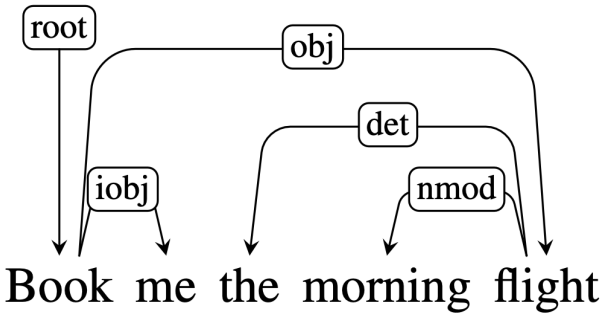
Assign *flight* as the head of *the*

Add relation

Parsing Example

Input sentence: Book me the morning flight

Goal:



Step	Stack	Buffer	Action	Relation Added
6	[root, book, the, morning, flight]	[]	Left-Arc	(morning ⟵ flight)
7	[root, book, the, flight]	[]	Left-Arc	(the ⟵ flight)
8	[root, book, flight]	[]	Right-Arc	(book → flight)

Assign *book* as the head of *flight*

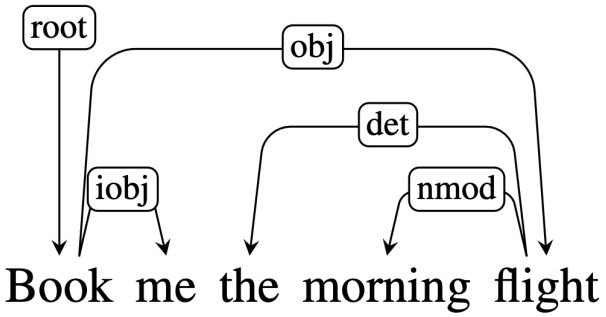
Add relation

Pop *flight* from the stack

Parsing Example

Input sentence: Book me the morning flight

Goal:



Step	Stack	Buffer	Action	Relation Added
6	[root, book, the, morning, flight]	[]	Left-Arc	(morning ⟵ flight)
7	[root, book, the, flight]	[]	Left-Arc	(the ⟵ flight)
8	[root, book, flight]	[]	Right-Arc	(book ⟶ flight)
9	[root, book]	[]	Right-Arc	(root ⟶ book)

Pop *book* from the stack

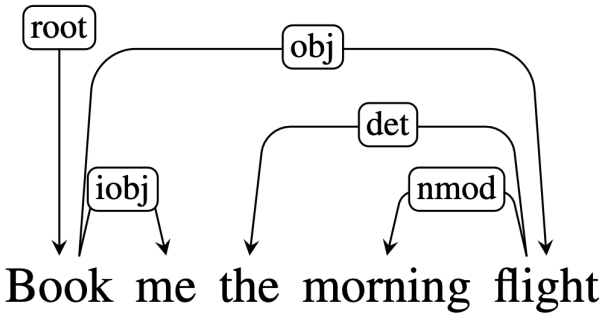
Assign *root* as the head of *book*

Add relation

Parsing Example

Input sentence: Book me the morning flight

Goal:



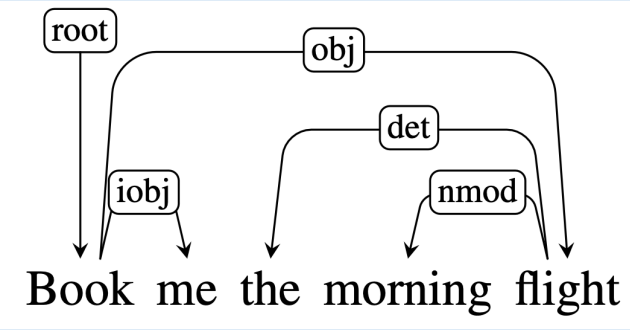
Step	Stack	Buffer	Action	Relation Added
6	[root, book, the, morning, flight]	[]	Left-Arc	(morning ⟵ flight)
7	[root, book, the, flight]	[]	Left-Arc	(the ⟵ flight)
8	[root, book, flight]	[]	Right-Arc	(book ⟶ flight)
9	[root, book]	[]	Right-Arc	(root ⟶ book)
10	[root]	[]	Done	

Parse is done when **stack** only contains **root** and **buffer** is empty

Train the Oracle

- Oracle's job: select the appropriate transition action based on the configuration: **stack (S)**, **buffer (B)**, and the already assigned **relations (R)**
- Trained by supervised machine learning
- **Training data**: **configurations** ($\boldsymbol{x}$) annotated with the correct **transition action** (y)
- ($\boldsymbol{x}$, y) are drawn from existing dependency trees

Treebank data:



How to convert?

Training data with paired ($\boldsymbol{x}$, y):

($\boldsymbol{x}_1$ = S_1, B_1, R_1 y_1 = Shift)

($\boldsymbol{x}_2$ = S_2, B_2, R_2 y_2 = Shift)

($\boldsymbol{x}_2$ = S_3, B_3, R_3 y_3 = Shift)

...

Procedure of Generating Training Data

- Given current configuration and a gold-standard **reference** parse
- Choose **Left-Arc** if:
  it produces a correct head-dependent relation
- Choose **Right-Arc** if:
  1) it produces a correct head-dependent relation **and**
  2) all dependents of the top word of stack have already been assigned
- Choose **Shift** otherwise

The restriction on Right-Arc is to prevent a word being popped from stack before all of its dependent have been assigned

Formally, given

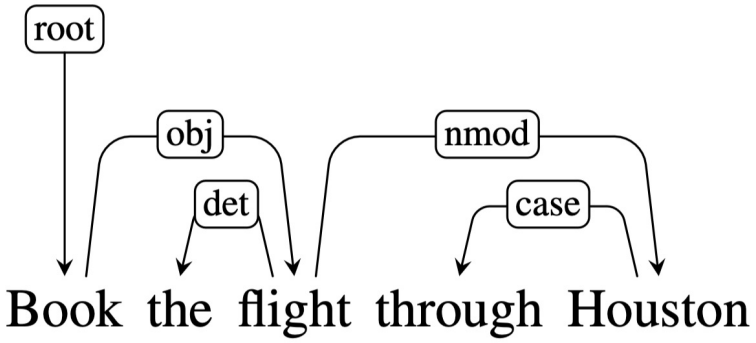
- a stack $\boldsymbol{S}$
- a reference parse with a set of relations $\boldsymbol{R}_{\text{ref}}$
- a set of currently assigned relations $\boldsymbol{R}_{\text{c}}$

Choose transition action as follows:

- $\text{Left}-\text{Arc}(r)$: **if** $\left(S_1 \overset{r}{\rightarrow} S_2\right) \in R_{\text{ref}}$
- $\text{Right}-\text{Arc}(r)$: **if** $\left(S_2 \overset{r}{\rightarrow} S_1\right) \in R_{\text{ref}}$ **and**
  $\forall r'$, w.r.t. $\left(S_1 \overset{r'}{\rightarrow} w\right) \in R_{\text{ref}} \Rightarrow \left(S_1 \overset{r'}{\rightarrow} w\right) \in R_{\text{c}}$
- Shift: **otherwise**

Procedure of Generating Training Data

- Gold-standard sentence: Book the flight through Houston
- with reference parse:



Relations in this reference parse

$$R_{\text{ref}} = \{\text{root} \rightarrow \text{book},$$
$$\text{book} \xrightarrow{\text{obj}} \text{flight},$$
$$\text{flight} \xrightarrow{\text{det}} \text{the},$$
$$\text{flight} \xrightarrow{\text{nmod}} \text{Houston},$$
$$\text{Houston} \xrightarrow{\text{case}} \text{through}\}$$

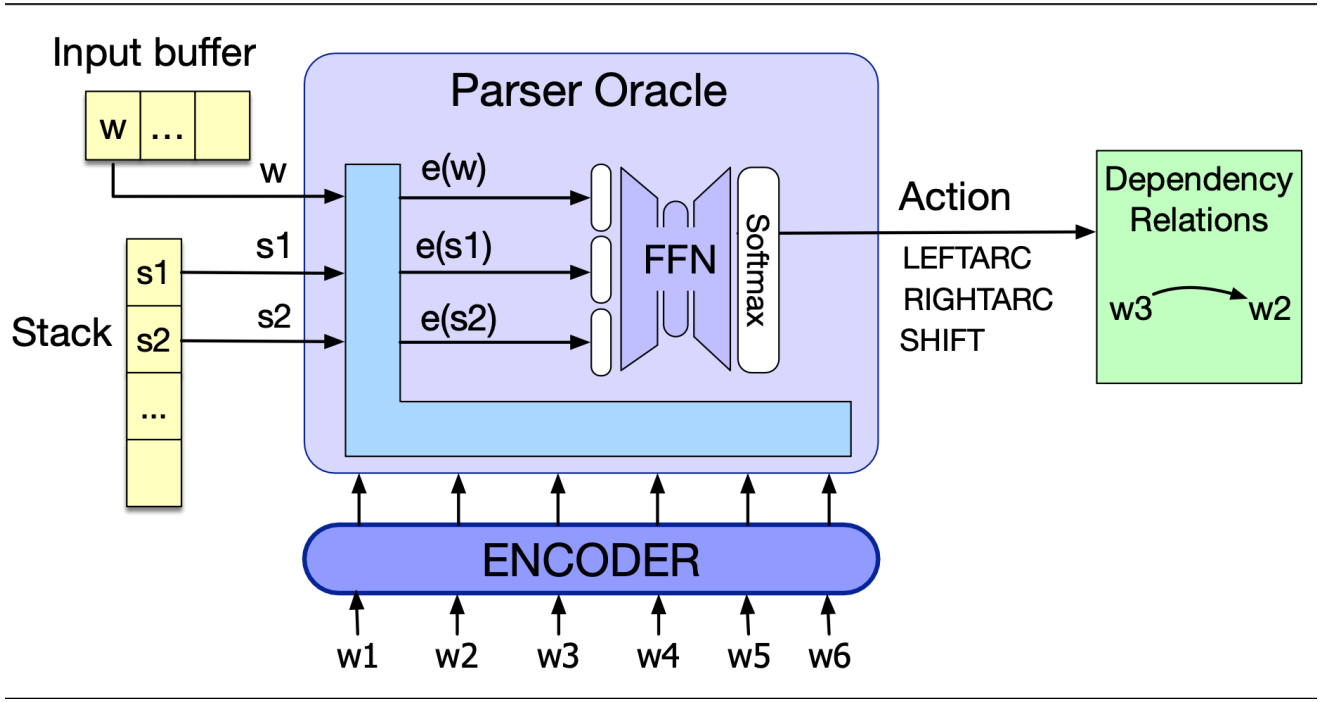
Initial configuration stage:
stack = [root], **buffer** = [book, the, …],
currently assigned relations $\boldsymbol{R_c} = \{\}$

Generated Data for Training

Step	Stack	Buffer	Action
0	[root]	[book, the, flight, through, Houston]	Shift
1	[root, book]	[the, flight, through, Houston]	Shift
2	[root, book, the]	[flight, through, Houston]	Shift
3	[root, book, the, flight]	[through, Houston]	Left-Arc
4	[root, book, flight]	[through, Houston]	Shift
5	[root, book, flight, through]	[Houston]	Shift
6	[root, book, flight, through, Houston]	[]	Left-Arc
7	[root, book, flight, Houston]	[]	Right-Arc
8	[root, book, flight]	[]	Right-Arc
9	[root, book]	[]	Right-Arc
10	[root]	[]	Done

Implementing the Oracle: A Neural Classifier

- **A standard architecture**:
- Pass the sentence through a neural encoder
- Take the vector representation of the top two words on stack and the first word on buffer, *concatenate* them
- Pass to a feedforward network to predict the transition action (learning done with cross-entropy loss)
- In practice, more features can be used



Reference: Chen and Manning, 2014; Kiperwasser and Goldberg, 2016; Kulmizev et al., 2019

A Feed-Forward Neural Implementation

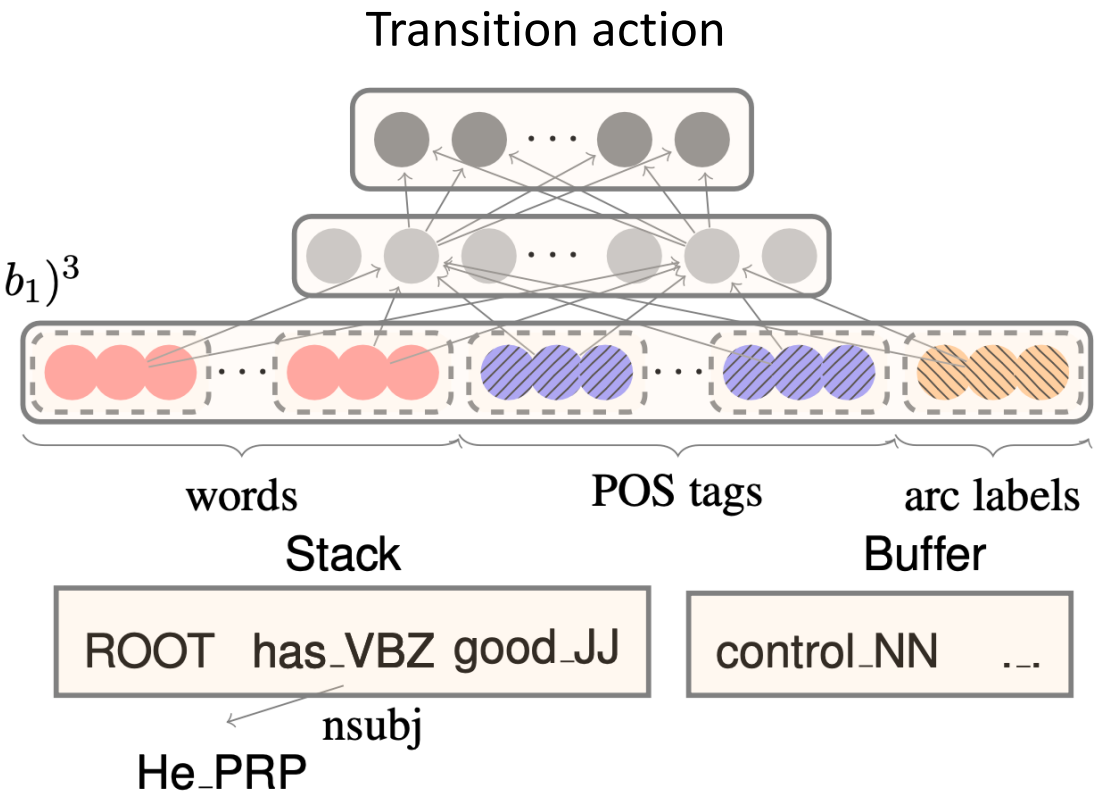
Softmax layer:

$$p = \texttt{softmax}(W_2 h)$$

Hidden layer:

$$h = (W_1^w x^w + W_1^t x^t + W_1^l x^l + b_1)^3$$

Input layer: $[x^w, x^t, x^l]$



Features used:

- The top 3 words on the stack and buffer, respectively: $s_0, s_1, s_2, b_0, b_1, b_2$
- The POS tags of the words
- …

Reference: Chen, D., & Manning, C. D. (2014, October). A fast and accurate dependency parser using neural networks.

Decoding/Inference Stage: Greedy Parsing

Algorithm 1 Greedy transition-based parsing

Input: sentence $s = w_1, \ldots, x_w, \ t_1, \ldots, t_n$, parameterized function $\text{SCORE}_\theta(\cdot)$ with parameters θ.
$c \leftarrow \text{INITIAL}(s)$
while not $\text{TERMINAL}(c)$ **do**
  $\hat{t} \leftarrow \arg\max_{t \in \text{LEGAL}(c)} \text{SCORE}_\theta(\phi(c), t)$
  $c \leftarrow \hat{t}(c)$
return $tree(c)$

Figure from Kiperwasser and Goldberg (2016)

- **TERMINAL**(c) can be manually decided (c for candidate)
- $t \in \text{LEGAL}(c)$ means that not all predictions from the oracle are legal transitions: Left-Arc or Right-Arc on empty stack; Left-Arc to root; etc.
- Only select from the top scored transition among the legal ones

Overview

- Dependency Grammars
- Transition-Based Dependency Parsing
- **Relation to Cognitive Measures**

General Principle

- Linguistic structure reflects the cognitive effort needed to understand a language.
- Higher structural **complexity** ⇒ Larger cognitive load（认知负载）
- Lower structural **complexity** ⇒ Smaller cognitive load

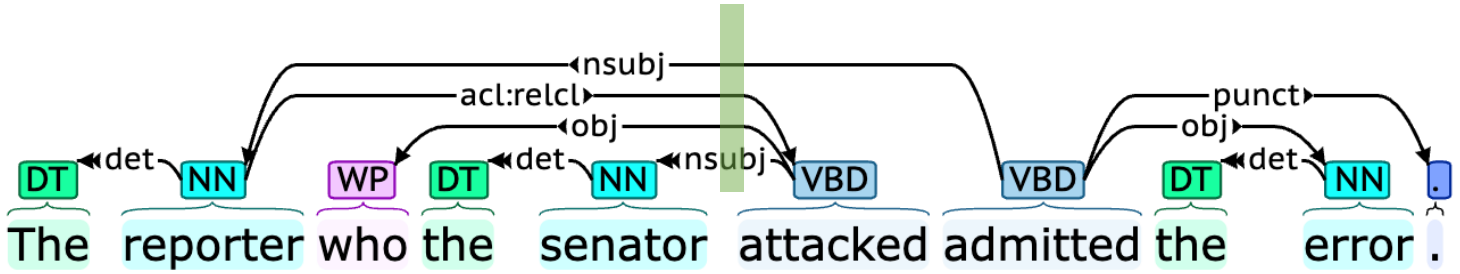
Source: Gibson (2000), https://doi.org/10.1016/S0010-0277(98)00034-1

Cognitive Effort in Processing Language

- Theory: **Locality** of syntactic dependencies
- `0: The reporter admitted the error.`
- `1a: The reporter who the senator attacked admitted the error.`
- `1b: The reporter who attacked the senator admitted the error.`
- Both relative clauses (RCs)
  - 1a => “who” is extracted from the **object** position of RC. => Hard
  - 1b => “who” is extracted from the **subject** position of RC. => Easy

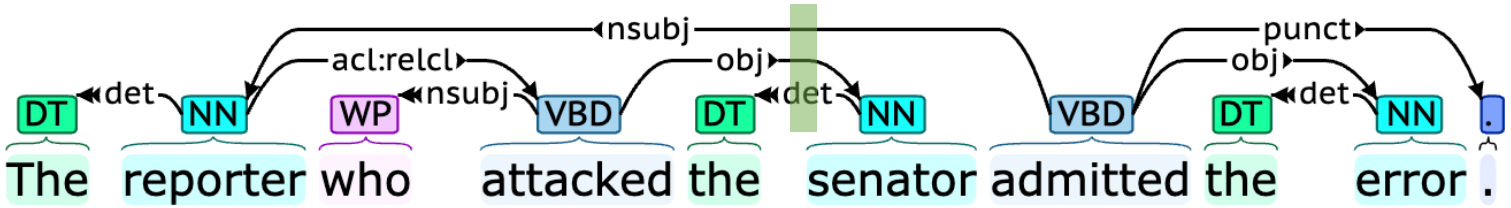
Dependency parsed sentences

- 1a: The reporter who the senator attacked admitted the error.



more stacked arrows

- 1

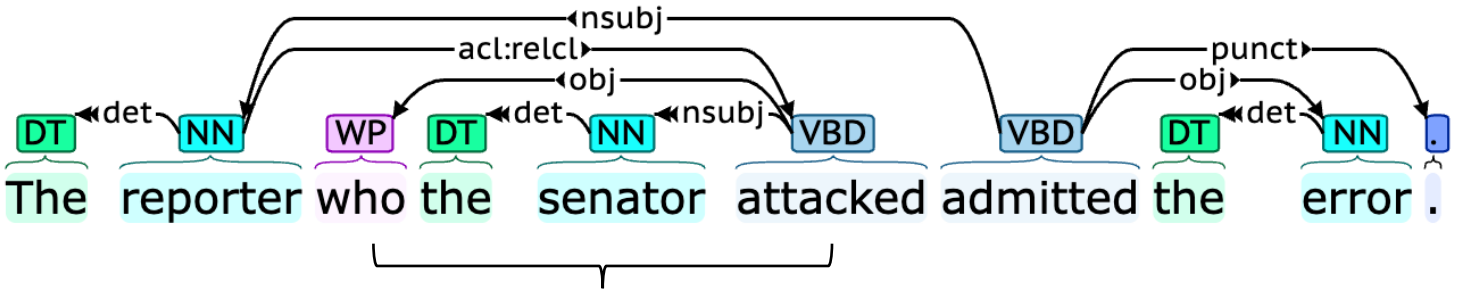


fewer stacked arrows

The memory costs are different

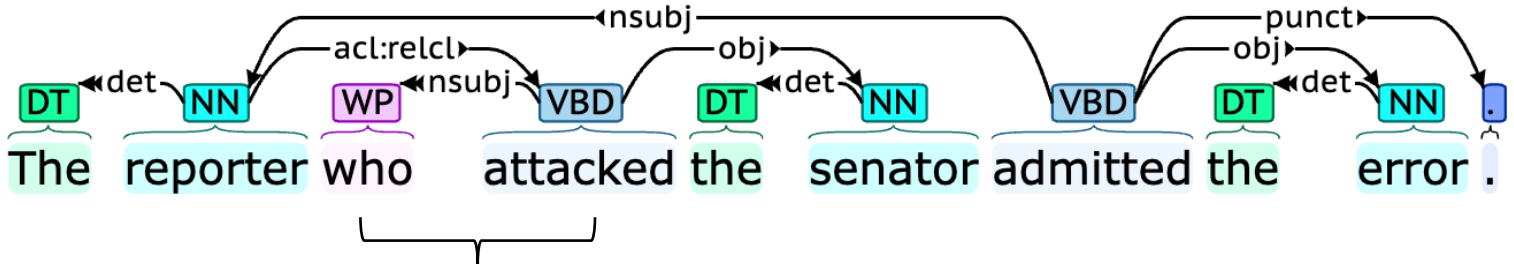
- One explanation: the distance between “who” and “attacked” determines the memory cost to resolve their dependency relation -- locality.

1a



less “local”, more memory cost to wait for “the senator” to be parsed.

1b



more “local”, less memory cost